Exploratory Data Analysis

Presenter: Ganesh Bharatwaj

IIT Madras, BS in Data Science and Applications

What is EDA?

- An approach of analyzing data sets to summarize their main characteristics.
- Term coined by John W. Tukey in the 1970s.
- It's not a formal set of rules; rather it is an "art" or "philosophy" of data investigation.
- Involves **data visualization** and **statistical analysis** techniques to help understand the data better.

Objectives of EDA

- Enable unexpected discoveries in the data
- Suggest hypotheses about the causes of observed phenomena
- Assess assumptions on which statistical inference will be based
- Support the selection of appropriate statistical tools and techniques
- Provide a basis for further data collection through surveys or experiments

Our Toolkit

- **Pandas** for **data and file manipulation**
- **Numpy** for **numerical computation**
- **Matplotlib** and **Seaborn** for **data visualization**
- **Jupyter Notebook** or **Google Colab** as **coding environment**



A Basic EDA Workflow

- Load and Inspect the Data
  - Get the data and observe its features, labels, and datatypes
- Clean the Data
  - Handling missing values, duplicates and outliers
- Univariate Analysis
  - Analyze one variable at a time
- Bivariate Analysis
  - Analyze two variables at a time
- Multivariate Analysis
  - Analyze three or more variables at a time

Loading and Inspecting the Data

- Load the data

```
import pandas as pd # importing Pandas library

# Loading the dataset (say file.csv)
df = pd.read_csv("file.csv")
```

- Viewing the data

```
# View the first 5 rows
df.head()
```

```
# View the last 5 rows
df.tail()
```

```
df.shape # (rows, columns)
```

- Show the data type of each column

```
df.info()
```

- Basic statistics of numerical columns

```
df.describe()
```

Cleaning the Data

- Identify and handle missing values

```
# gives total number of missing values for each column
```

```
df.isnull().sum()
```

```
df.isna().sum()
```

```
# Handling missing values
```

```
df.fillna() # impute with mean, median, mode, standard deviation
```

- Identify and handle duplicates

```
# Finding duplicates
```

```
df.duplicated().sum()
```

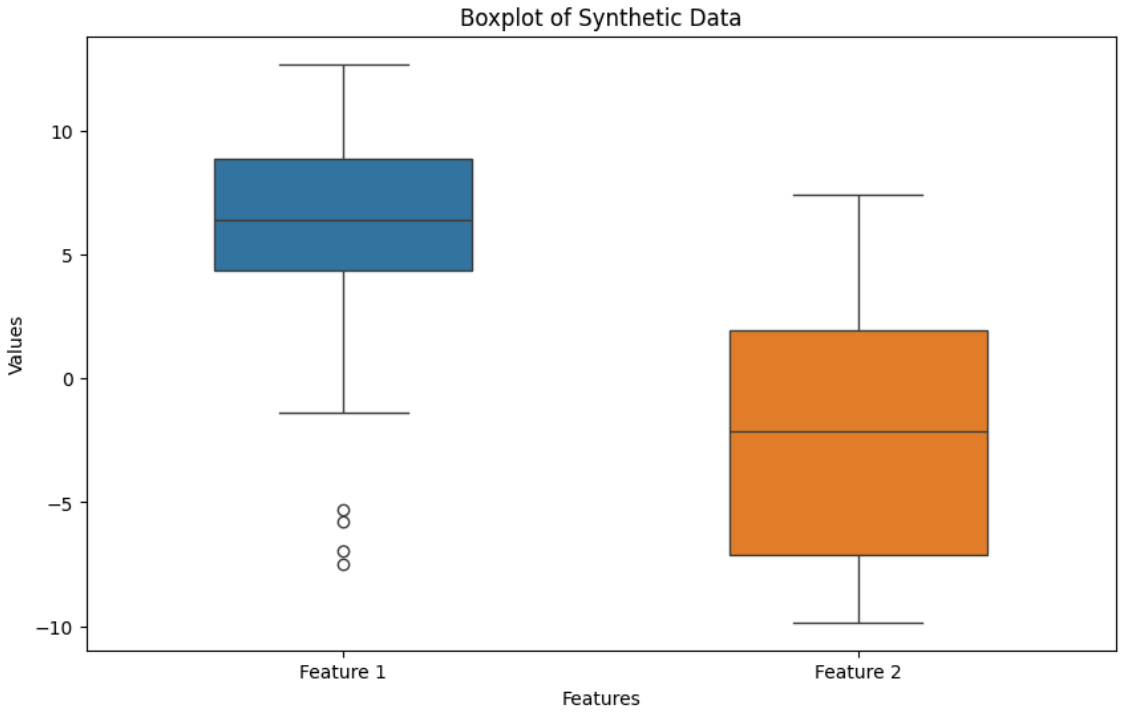
```
# Dropping duplicates
```

```
df.drop_duplicates()
```

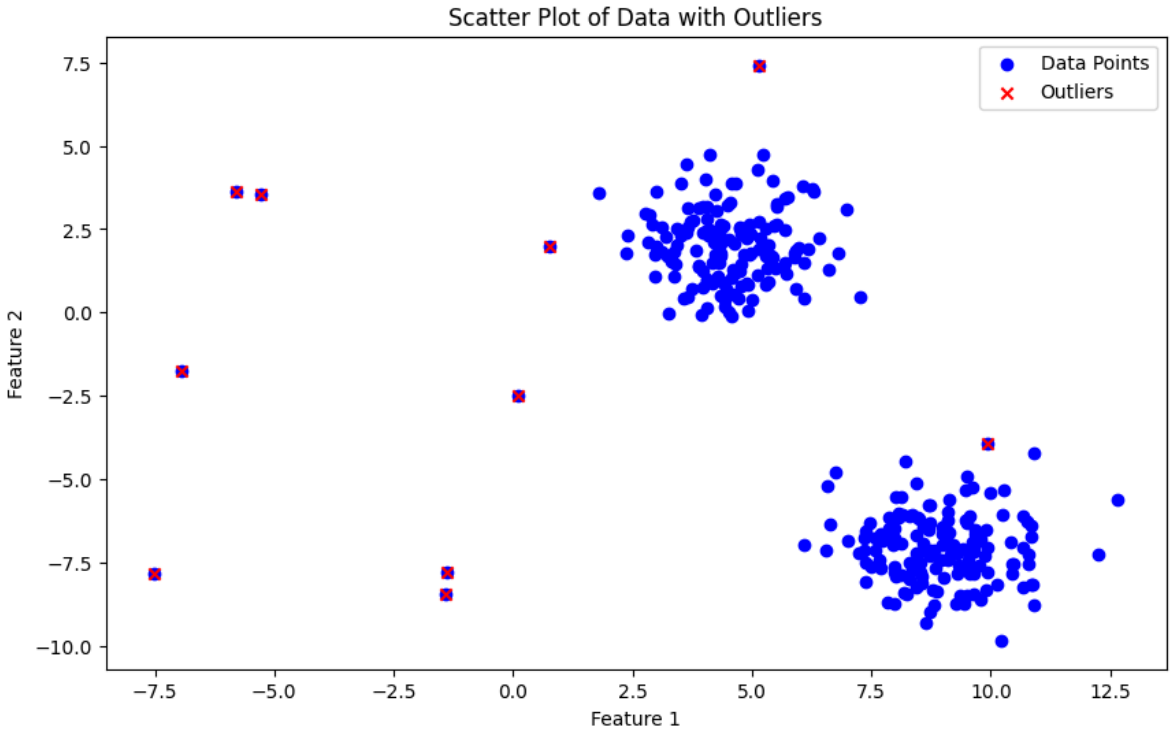
- Identify and handle outliers
  - Better done by data visualization, such as analyzing box plots

Identifying Outliers

- Visual methods
  - Box plots: points outside the "whiskers" are outliers
  - Scatter plots: points outside the main cluster(s) are outliers



Identifying Outliers (continued)



Identifying Outliers - Statistical Methods

- Z-score
  - Measures how many standard deviations (σ) a point is from the mean (μ).

$$Z = \frac{x - \mu}{\sigma}$$

$Z > 3$ is a common threshold for outliers

- IQR Score

$$IQR = Q_3 - Q_1$$

$x < Q_1 - 1.5 \cdot IQR$ or $x > Q_3 + 1.5 \cdot IQR \implies$ outliers

Identifying Outliers - Machine Learning Methods (Advanced)

- Isolation Forest
  - An unsupervised learning, decision tree based algorithm
  - Isolates observations by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature
  - The recursive partitioning is represented by a tree structure, and the number of splittings required to isolate a sample is equivalent to the path length from the root node to the terminating node.
  - This path length, averaged over a forest of such random trees, is a measure of normality and our decision function.
  - Random partitioning produces noticeably shorter paths for anomalies. Hence, when a forest of random trees collectively produce shorter path lengths for particular samples, they are highly likely to be outliers.
- Local Outlier Factor (LOF)
  - Unsupervised anomaly detection method.
  - Computes the local density deviation of a given data point with respect to its neighbors.
  - Samples that have a substantially lower density than their neighbors are considered as outliers.

Handling Outliers

- Discard or correct the values
  - Some outliers might be mistakes or incorrect values.
  - Some outliers are simply noise (but not always - be careful)
- Keep them
  - Sometimes, they can convey serious information for training a model in later stages.
- Transform or cap (Winsorize) them
  - e.g. Log Transformation: $x \longrightarrow \ln(1+x)$
  - e.g. Winsorization: Set all values above the 99th percentile *to* the 99th percentile value.

Univariate Analysis (Numerical)

"What is the distribution of this variable (feature)?"

- **Histogram**: Shows the **frequency distribution**

```
import matplotlib.pyplot as plt
import seaborn as sns

X, y = df.drop('label', axis=1), df['label']

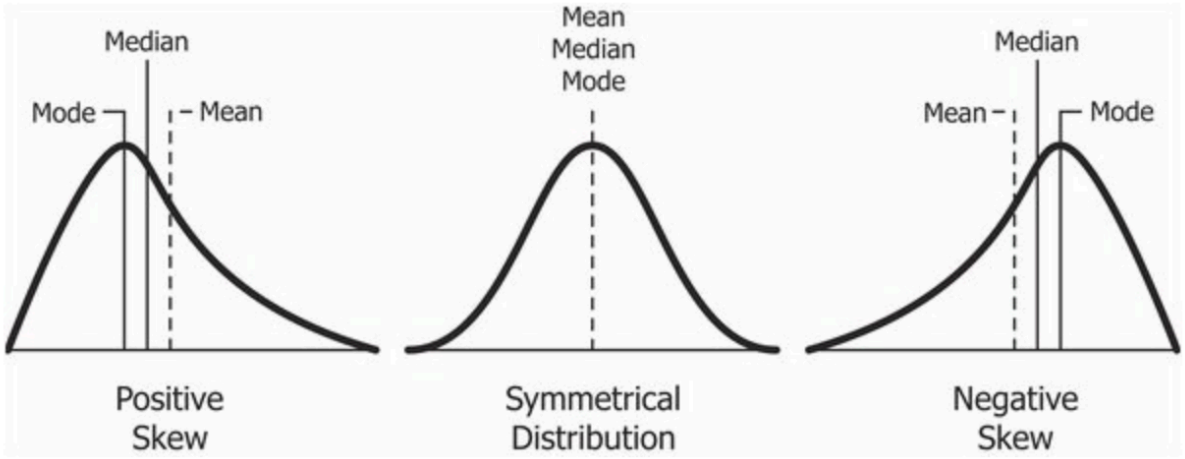
sns.histplot(data=X)
plt.show()
```

- **Boxplot**: Shows **outliers** and gives a **5-number summary** (min, Q_1, median, Q_3, max) of the data

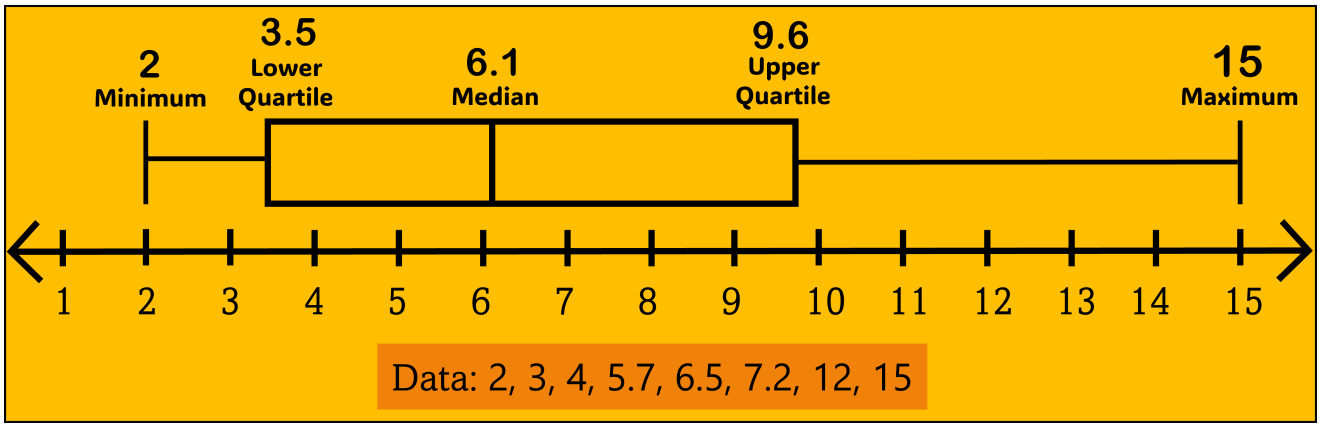
```
sns.boxplot(data=X)
plt.show()
```

Skewness

- **Skewness** is a measure of the **asymmetry** of the probability distribution of a random variable about its **mean**.



Box Plot


2
Minimum
3.5
Lower
Quartile
6.1
Median
9.6
Upper
Quartile
15
Maximum
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Data: 2, 3, 4, 5.7, 6.5, 7.2, 12, 15

Univariate Analysis (Categorical)

"What are the counts of each category?"

```
df['categorical_column'].value_counts()
```

```
# Bar Chart or Count Plot
categorical_data = df['categorical_column']
sns.countplot(data=categorical_data)
```

```
# Visualizing Categorical Data (careful)
categorical_data = df['categorical_column']
# Pie chart for visualization
plt.pie(categorical_data)
plt.show()
```

Generally advisable to avoid visualization if there are too many categories.

Bivariate Analysis (Numerical vs Numerical)

- To observe how numerical features are correlated with each other.
- Scatter Plot

```
sns.scatterplot(data=X, x='numerical_feature_1', y='numerical_feature_2')
```

- Correlation Heatmap

```
import numpy as np
# Selecting all numerical features
numerical_features = X.select_dtypes(include=np.number).columns.tolist()
X_numerical = X[numerical_features]
# Plotting annotated correlation heatmap
sns.heatmap(X_numerical.corr(), annot=True)
```

- **Remember:** Correlation does not imply causation!

Bivariate Analysis (Numerical vs Categorical)

"Does the numerical distribution change across different categories?"

- Box Plot (grouped)
  - Example: Comparing the Price (numeric) of houses across different Neighborhoods (categorical).

```
sns.boxplot(data=X, x="categorical_column", y="numerical_column")
```

- Violin Plot
  - It shows the distribution of data points after grouping by one (or more) variables. Unlike a box plot, each violin is drawn using a kernel density estimate (KDE) of the underlying distribution.

```
sns.violinplot(data=X, x="categorical_column", y="numerical_column")
```

Bivariate Analysis (Categorical vs Categorical)

"How are the counts of one category distributed across another?"

- Code:

```
pd.crosstab(X['categorical_1'], X['categorical_2']
```

- Visualizations:
  - Stacked Bar Chart
  - Grouped Bar Chart

```
# Loading the Titanic dataset
data = sns.load_dataset("titanic")
# Showing Survived vs. Not Survived broken down by Gender in the Titanic dataset.
sns.countplot(data=data, x='survived', hue='sex')
```

Multivariate Analysis

"How do multiple variables interact?"

- Pair Plot (very powerful)
  - Creates a grid of scatter plots for every (Numerical vs. Numerical) pair and histograms for each variable.

```
penguins = sns.load_dataset("penguins")
sns.pairplot(penguins)
```

  - Add a category with hue:

```
sns.pairplot(penguins, hue="species")
```

- Scatter Plot with hue or size:

```
tips = sns.load_dataset("tips")
sns.scatterplot(data=tips, x="total_bill", y="tip", hue="size", size="size")
```

  - This lets you visualize 4 variables at once! (x-axis, y-axis, color, and dot size).

Conclusion

- EDA is to be treated as an **iterative cycle** rather than a linear checklist.
- **Keep asking questions**, and generate visualizations and statistical inferences to answer those questions.
- **Document your findings**, interpretations and inferences in your notebook. This will prove useful in later stages such as feature engineering, data preprocessing, as well as training a machine learning model.

Thank You!